

OddsWatch

Real-Time Sports Betting Odds & Line-Movement Platform

MGMT 59000 - Cloud Data Engineering
Final Project Deliverables

Danny Affleck

July 2026

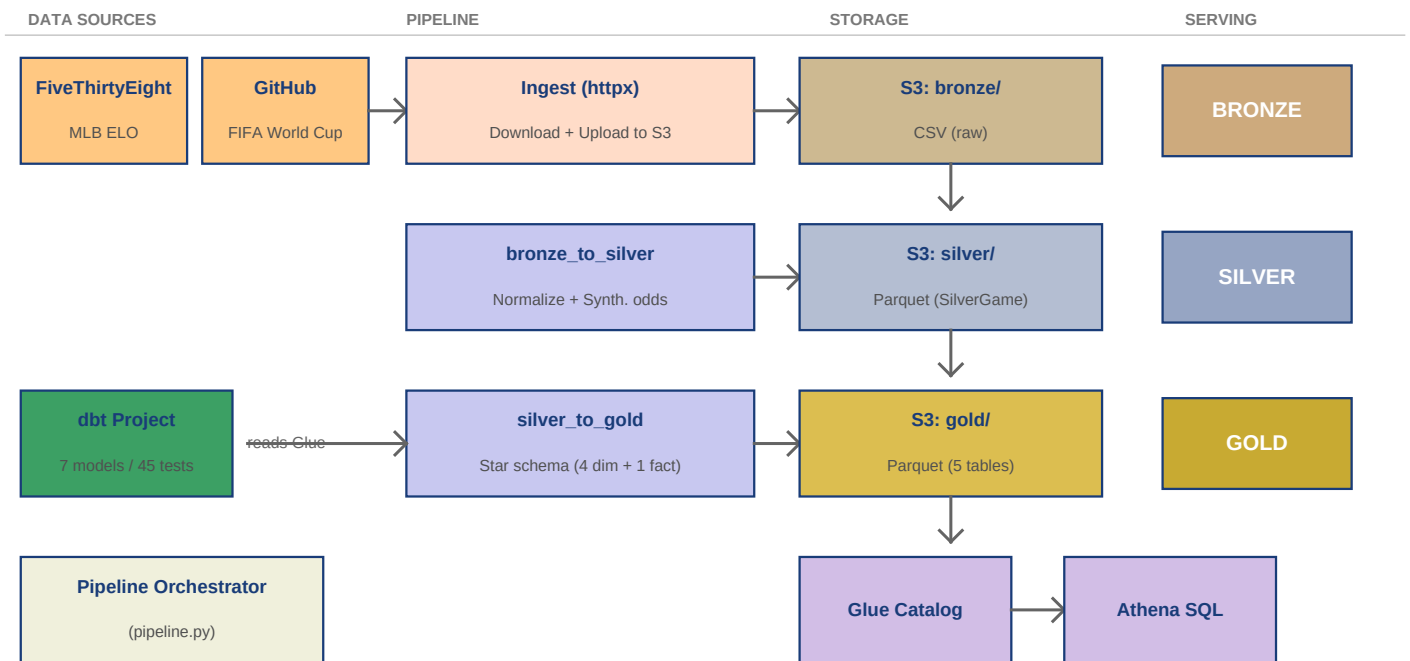
1. Architecture Diagram & Pattern Justification

1.1 Pattern: Medallion Lakehouse Architecture

OddsWatch follows a Medallion Lakehouse architecture (Bronze / Silver / Gold) running on AWS. This pattern was chosen because the project ingests semi-structured data from heterogeneous public sources (MLB ELO ratings and FIFA World Cup results), progressively refines it through normalization and enrichment stages, and ultimately serves it as a star schema optimized for analytical queries. The Lakehouse pattern combines the low-cost scalability of a data lake (S3 + Parquet) with the schema enforcement and catalog capabilities of a data warehouse (Glue Data Catalog), making it ideal for a batch-oriented analytics pipeline that may later add a streaming layer.

1.2 Architecture Diagram

The following diagram shows the end-to-end data flow through the OddsWatch system. Each box represents a component in the pipeline; arrows indicate data movement.



Format: CSV (bronze) | Parquet with Snappy (silver, gold) Catalog: AWS Glue Data Catalog Pattern: Medallion Lakehouse

1.3 Component Summary

Ingestion (httpx)

Downloads raw data from FiveThirtyEight (MLB ELO) and GitHub (FIFA World Cup results) via HTTP, uploads as CSV to S3 bronze layer.

Bronze Layer (S3)

Raw CSV files stored unmodified. Source of truth for replay.

Silver Transform

Normalizes two heterogeneous schemas into a unified SilverGame model. Generates synthetic closing odds (spread, total, moneyline) seeded for reproducibility.

Gold Transform

Builds a star schema: 4 dimension tables (DimTeam, DimGame, DimDate, DimMarket) and 1 fact table (FactGameOdds) with derived metrics (cover, over/under result).

S3 Storage

All silver and gold data stored as Parquet (Snappy compression) for columnar query performance.

Glue Data Catalog

Registers all tables as `EXTERNAL_TABLEs` with column schemas, enabling SQL access via Athena.

Pipeline Orchestrator

`pipeline.py` coordinates the full end-to-end run: ingest, transform (bronze-to-silver, silver-to-gold), upload, and catalog registration.

1.4 Why Lakehouse Over Alternatives

Lambda Architecture was considered but rejected because OddsWatch currently runs as a batch-only pipeline. Maintaining separate batch and speed layers would add complexity with no benefit until real-time odds streaming is implemented. Data Mesh was also considered but is premature for a two-source, single-team project. The Lakehouse pattern gives us the flexibility to add a streaming layer (Kinesis / Kafka) alongside the batch layer later, while keeping a single source of truth in S3.

2. Data Model, Schema & SCD Strategy

2.1 Three-Layer Schema Overview

Data progresses through three layers. Each layer has a distinct purpose: Bronze preserves raw fidelity, Silver normalizes and enriches, and Gold optimizes for analytical queries via a star schema.

2.2 Bronze Layer (Raw Ingestion)

Two source-specific schemas, stored as CSV. No transformations applied.

BronzeMLBGame (8 columns)

Column	Type	Description
date	str	Game date (YYYY-MM-DD)
season	int	Season year
neutral	int	1 if neutral site, 0 otherwise
playoff	str None	Playoff round indicator or None
team1	str	Home team abbreviation
team2	str	Away team abbreviation
score1	float	Home team score
score2	float	Away team score

BronzeWorldCupMatch (9 columns)

date	str	Match date (YYYY-MM-DD)
home team	str	Home team name
away team	str	Away team name
home score	int	Home team goals
away score	int	Away team goals
tournament	str	Tournament name (filtered to FIFA World Cup)
city	str	Host city
country	str	Host country
neutral	str	Neutral venue flag

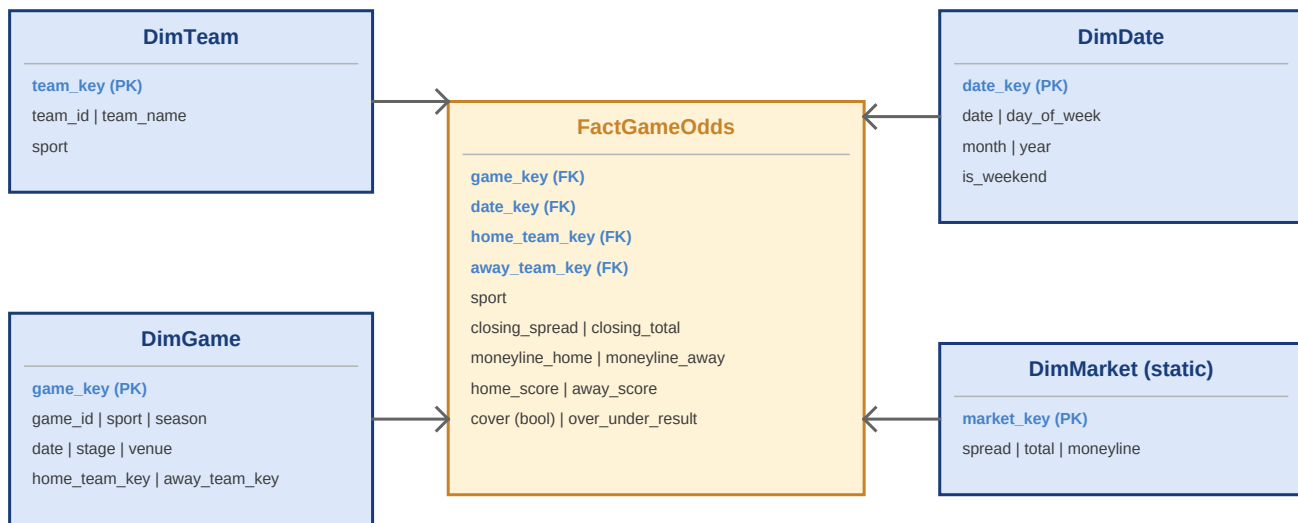
2.3 Silver Layer (Unified & Enriched)

Both sources are normalized into a single SilverGame schema (14 columns). Synthetic closing odds are generated deterministically from actual game outcomes using seeded random noise, rounded to half-point precision.

Column	Type	Description
game_id	str	Unique ID: {sport} {date} {team1} {team2} {idx}
sport	str	Sport identifier (mlb or world_cup)
date	date	Game date as Python date object
season	int	Season/tournament year
home_team	str	Home team name (uppercase normalized)
away_team	str	Away team name (uppercase normalized)
home_score	int	Home team final score
away_score	int	Away team final score
stage	str None	Game stage (playoff, regular_season, or None)
venue	str None	Venue or city (None for MLB)
closing_spread	float	Synthetic point spread (half-point rounded)
closing_total	float	Synthetic over/under line (min 0.5)
closing_moneyline_home	int	American moneyline odds for home team
closing_moneyline_away	int	American moneyline odds for away team

2.4 Gold Layer (Star Schema)

The gold layer implements a star schema with four dimension tables surrounding one central fact table. This design optimizes for analytical queries such as 'What percentage of home favorites covered the spread on weekends?'



Star schema: 4 dimension tables surround 1 central fact table | 13 fact columns | Derived metrics: cover, over_under_result

2.4.1 Gold Table Column Details

DimTeam

Column	Type	Description
team key	int	Surrogate key (sequential)
team id	str	Derived ID (lowercase, underscores)
team name	str	Display name
sport	str	Sport (mlb or world cup)
group or conference	str None	Reserved for future enrichment
country	str None	Reserved for future enrichment

DimGame

Column	Type	Description
game key	int	Surrogate key
game id	str	Natural key from silver layer
sport	str	Sport identifier
season	int	Season year
date	date	Game date
stage	str None	Playoff/regular season/None
home team key	int	FK to DimTeam
away team key	int	FK to DimTeam
venue	str None	Venue name or city

DimDate

Column	Type	Description
date key	int	Surrogate key
date	date	Calendar date
day of week	str	Day name (Monday, Tuesday, ...)
month	int	Month number (1-12)
year	int	Calendar year
is weekend	bool	True if Saturday or Sunday

DimMarket (static, 3 rows)

Column	Type	Description
market key	int	Surrogate key (1, 2, 3)
market type	str	spread / total / moneyline
description	str	Human-readable label

FactGameOdds (central fact table)

Column	Type	Description
game key	int	FK to DimGame
date key	int	FK to DimDate
home team key	int	FK to DimTeam
away team key	int	FK to DimTeam
sport	str	Sport degenerate dimension
closing spread	float	Closing point spread
closing total	float	Closing over/under line
closing moneyline home	int	Home moneyline odds
closing moneyline away	int	Away moneyline odds
home score	int	Home team final score

away_score	int	Away team final score
cover	bool	True if home team covered spread
over_under_result	str	over / under / push

2.5 Slowly Changing Dimension (SCD) Strategy

The following SCD strategies are applied to each dimension table:

Dimension	SCD Type	Rationale
DimTeam	Type 1	Team names/abbrevs rarely change. Overwrite on refresh; no need to track historical names for odds analysis.
DimGame	Type 0	Game facts (date, score, teams) are immutable once recorded. No updates expected.
DimDate	Type 0	Calendar attributes are fixed and deterministic. Never changes.
DimMarket	Type 0	Static reference table with 3 rows. Market types do not change.
DimBook*	Type 2	Sportsbook fee structures and odds formats change over time. valid_from / valid_to / is_current fields track history.

* DimBook is defined in the data model for future streaming use; not populated in the current batch pipeline.

In the current batch pipeline, all gold tables are rebuilt from scratch on each run (full refresh). This is appropriate given the data volume (~200K rows total) and batch cadence. When the streaming layer is added, incremental upsert logic with SCD Type 2 tracking will be applied to DimBook via the valid_from/valid_to/is_current fields already defined in the Pydantic model.

3. AWS vs. GCP Service Mapping Memo

This memo maps each pipeline component to its AWS service (used in the current implementation) and the equivalent GCP service. The final column provides the rationale for choosing AWS.

Component	AWS (Current)	GCP Equivalent	Rationale for AWS
Object Storage	Amazon S3	Cloud Storage	S3 is the de facto standard for data lakes. Native integration with Glue, Athena, and the broader AWS analytics stack.
Data Catalog	AWS Glue Catalog	Data Catalog	Glue Catalog provides Hive-compatible metastore that Athena queries directly. No separate metastore to manage.
ETL / Transform	AWS Glue Jobs	Dataflow / Dataproc	Glue supports serverless Spark and Python shell jobs. Current pipeline uses Python scripts that can be deployed as Glue Python Shell jobs.
SQL Analytics	Amazon Athena	BigQuery	Athena is serverless, pay-per-query, and reads directly from S3+Glue Catalog. No data loading step required.
Orchestration	Step Functions	Cloud Composer	Step Functions integrates natively with Glue and Lambda for serverless orchestration of the batch pipeline.
Compute	AWS Lambda	Cloud Functions	Lambda handles lightweight ingestion tasks (HTTP downloads, S3 uploads) within the 15-min/10GB limits.
Streaming*	Kinesis Data Streams	Pub/Sub	Kinesis integrates with Glue Streaming and Lambda for future real-time odds ingestion.
IAM & Security	AWS IAM	Cloud IAM	IAM roles and policies control access to S3 buckets, Glue databases, and Athena workgroups.

* Streaming component is planned for future implementation.

3.1 Why AWS Over GCP

AWS was selected for three reasons: (1) the S3 + Glue + Athena stack provides a tightly integrated, serverless analytics platform that requires minimal infrastructure management; (2) the course curriculum is built around AWS services; and (3) the boto3 SDK provides mature Python support for all services used. GCP's BigQuery offers superior query performance on large datasets but requires data loading, which adds complexity for a pipeline that already stores data in a lakehouse format.

4. Cloud Cost Estimate

Back-of-envelope cost estimate based on current data volumes and a daily batch pipeline cadence. All prices are us-east-1 on-demand rates as of mid-2026.

4.1 Data Volume Assumptions

- MLB ELO dataset: ~230,000 rows, ~12 MB CSV, ~3 MB Parquet
- World Cup dataset: ~1,000 rows, ~80 KB CSV, ~25 KB Parquet
- Gold layer (all 5 tables): ~5 MB Parquet total
- Total S3 footprint: ~20 MB (all layers combined)
- Pipeline runs: 1x daily (batch)

4.2 Monthly Cost Breakdown

Service	Usage	Unit Price	Monthly Cost
S3 Storage	~20 MB	\$0.023/GB	\$0.01
S3 PUT Requests	~300 PUTs/month	\$0.005/1K	\$0.01
S3 GET Requests	~900 GETs/month	\$0.0004/1K	\$0.01
Glue Catalog Storage	12 tables	Free (< 1M)	\$0.00
Glue API Requests	~360 calls/month	\$1.00/1M	\$0.01
Athena Queries	~100 queries, 5 MB ea	\$5.00/TB scanned	\$0.01
Lambda (Ingestion)	30 invocations, 30s ea	\$0.20/1M + compute	\$0.01
Data Transfer	~50 MB out/month	First 100 GB free	\$0.00

Estimated Total: < \$1.00 / month

At current volumes, costs are negligible and fall within AWS Free Tier limits for the first 12 months. The dominant cost driver at scale would be Athena query scanning and S3 storage, both of which are addressed in the scale analysis below.

4.3 Cost at 10x Scale

At 10x data volume (~200 MB total storage, ~2.3M rows), monthly costs remain under \$5.00. Athena becomes the primary cost driver at \$5/TB scanned. Partitioning gold tables by sport and season would reduce scan volume by 50-70%, keeping Athena costs proportional to the data actually queried rather than total stored. At 100x (~2 GB, 23M rows), estimated cost rises to ~\$15-25/month, still very manageable for an analytics workload.

5. Scale Analysis: What Breaks at 10x

This section analyzes what components of the OddsWatch pipeline would degrade or fail if data volume increased by 10x (from ~230K rows to ~2.3M rows) and how each bottleneck can be resolved.

5.1 Ingestion Layer

Current: Single-threaded HTTP download via httpx

Risk: At 10x, the raw CSV download from FiveThirtyEight grows from ~12 MB to ~120 MB. A single HTTP GET remains feasible but takes ~30-60 seconds on a typical connection. At 100x, API rate limits and download times become blocking.

Mitigation:

- Paginate or chunk large downloads using Range headers or API pagination.
- Parallelize ingestion across multiple data sources using async httpx.
- Add incremental ingestion: track last-ingested date, only fetch new records.

5.2 Transform Layer

Current: In-memory pandas DataFrames, single-process

Risk: The bronze-to-silver transform iterates row-by-row to generate synthetic odds (Python for-loop with random seeding per row). At 2.3M rows, this loop becomes the primary bottleneck (~5-10 minutes). The silver-to-gold transform uses vectorized pandas operations and scales linearly; it remains feasible at 10x.

Mitigation:

- Vectorize synthetic odds generation using numpy.random with array-level seeds instead of per-row seeding, reducing O(n) Python loop to O(1) numpy call.
- At 100x, migrate to PySpark on Glue for distributed processing.
- Partition output Parquet by sport and season for parallel write.

5.3 Storage Layer

Current: Single Parquet file per table, no partitioning

Risk: Single-file writes work at 20 MB but become inefficient at 200 MB+. Athena must scan entire files even for queries targeting a single sport or season. No partition pruning is possible.

Mitigation:

- Introduce Hive-style partitioning: gold/fact_game_odds/sport=mlb/season=2024/
- Use PyArrow write_to_dataset() with partition_cols for automatic partitioning.
- Register partition keys in Glue Catalog for Athena partition pruning.

5.4 Catalog & Query Layer

Current: Glue Catalog with unpartitioned tables

Risk: Athena scans all data for every query. At 10x, a simple aggregation query scans ~200 MB (\$0.001 per query). Costs are still low but query latency increases to 5-10 seconds. At 100x, full scans become both slow and expensive.

Mitigation:

- Add partition columns to Glue table definitions for partition pruning.

- Use Parquet columnar projection (Athena only reads referenced columns).
- Consider converting to Iceberg table format for ACID transactions and time-travel queries at warehouse scale.

5.5 Orchestration

Current: Single-function pipeline.py running locally

Risk: The monolithic pipeline.py runs all stages sequentially in a single process. At 10x, a failure in the gold transform requires re-running the entire pipeline from ingestion. No checkpointing or retry logic exists.

Mitigation:

- Deploy as AWS Step Functions state machine with per-stage error handling and retry policies.
- Add idempotency: each stage checks for existing output before re-processing.
- Enable parallel execution of independent stages (e.g., MLB and World Cup ingestion can run concurrently).

5.6 Summary Table

Component	Breaks At	Symptom	Fix
Ingestion	100x (120+ MB)	Timeout / OOM	Chunked + async
Transform	10x (2.3M rows)	Slow row-by-row loop	Vectorize / Spark
Storage	10x (200+ MB)	No partition pruning	Hive partitioning
Catalog/Query	10x (200+ MB scans)	Slow queries, cost	Partition + Iceberg
Orchestration	10x	No retry, full re-run	Step Functions

6. Trade-Off Analysis: Dominant Dimension

Per the Session 7 framework, every system has a dominant design dimension. The six dimensions from the course are: Caching, CAP Theorem, Fan-out, Geo-distribution, Streaming, and combined (Everything). For OddsWatch, the analysis is as follows:

6.1 Dominant Dimension: Streaming

The core value proposition of OddsWatch is tracking odds movement over time. While the current implementation is batch-only, the system is designed to evolve toward real-time streaming of odds ticks from multiple sportsbooks. The data model already includes FactOddsTick and DimBook tables for this purpose. As the platform matures, the dominant challenge is ingesting, windowing, and serving high-frequency odds updates with low latency. This makes Streaming the dominant dimension.

6.2 Dimensions Sacrificed

1. Geo-distribution (Sacrificed)

OddsWatch runs entirely in us-east-1. Sports betting odds are not latency-sensitive for analytics (millisecond edge matters for trading, not for historical analysis). Multi-region replication would add cost and complexity with no analytical benefit.

2. Fan-out (Sacrificed)

The system has a single producer (the pipeline) and a small number of consumers (analysts running Athena queries). There is no fan-out challenge: no push notifications, no timeline assembly, no follower graph. Queries are pull-based and ad-hoc. This eliminates the need for caching layers, CDNs, or pre-computed materialized views.

6.3 Dimensions Preserved (But Not Dominant)

CAP: Eventual Consistency Accepted

The batch pipeline prioritizes availability (the pipeline always completes) over strict consistency (gold tables may be briefly stale during a run). This is acceptable because analysts query completed snapshots, not in-flight data.

Caching: Not Required Today, Natural Fit Later

At current query volumes, Athena scans are fast and cheap. If a front-end dashboard is added, a Redis or ElastiCache layer for hot queries would be the first optimization, but it is not a current design constraint.